

Relaciones entre clases

Objetivo de las relaciones

- Representan la forma en que los objetos **interactúan** entre sí o **dependen** unos de otros dentro de un sistema.
- Salvo casos excepcionales, no existen clases que actúen de forma aislada en un sistema.
- **Objetivo principal:** las relaciones existen para que las clases cumplan el objetivo/fin del sistema mediante la interacción.

¿Por qué las usamos?

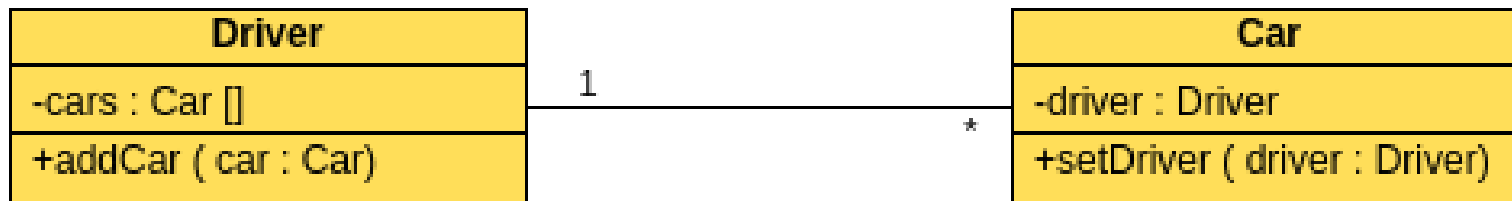
- **Modelan el comportamiento del sistema:** reducen el margen de error
- **Facilitan la construcción de sistemas robustos:** los cambios en una parte del sistema no afectan a otra parte.
- **Permiten que el diseño sea modular y reutilizable:** división de responsabilidades, bajo acoplamiento, alta cohesión
- **Completan la pieza faltante de los diagramas UML**

Tipos de relaciones

- En la especificación de UML contamos con 6 tipos de relaciones posibles entre clases.
- Los tipos de relaciones disponibles son:
 - Asociación
 - Agregación (variante de Asociación)
 - Composición (variante de Asociación)
 - Generalización (Herencia)
 - Realización (Implementación)
 - Dependencia

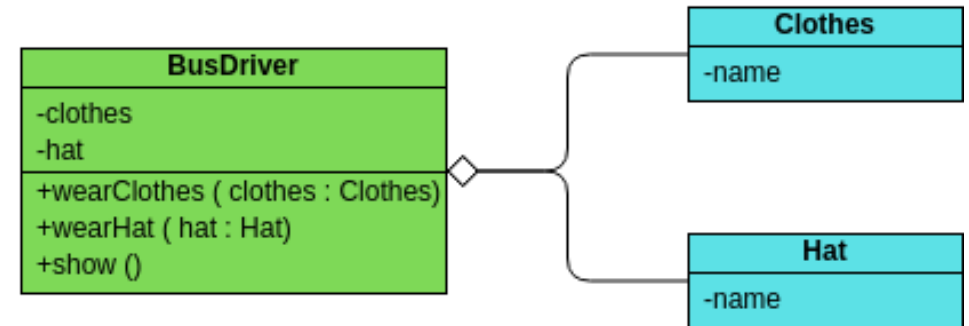
Asociación

- Indica que **una propiedad de una clase contiene una referencia a una instancia (o instancias) de otra clase.**
- La asociación es la relación **más utilizada** entre una clase y otra clase, lo que significa que existe una conexión entre un tipo de objeto y otro tipo de objeto. **Las composiciones y agregaciones también pertenecen a las relaciones asociativas**, pero esas relaciones son más fuertes que la asociación común (o de “afiliación”).
- Se indica la relación con una línea continua (con cardinalidad opcional).



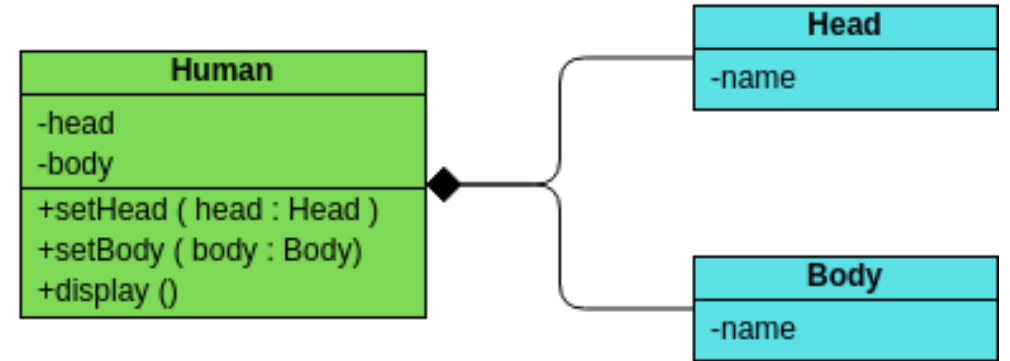
Agregación

- Relación de tipo "todo/parte", en la cual una clase contiene a otra, pero las instancias de las clases involucradas **pueden existir de manera independiente**.
- La clase que contiene es conocida como la clase "todo" y la clase contenida como la clase "parte".
- En la agregación, la vida de las instancias contenidas no depende de la existencia de la clase contenedora.
- Se indica la relación con una línea continua con una punta de rombo blanco



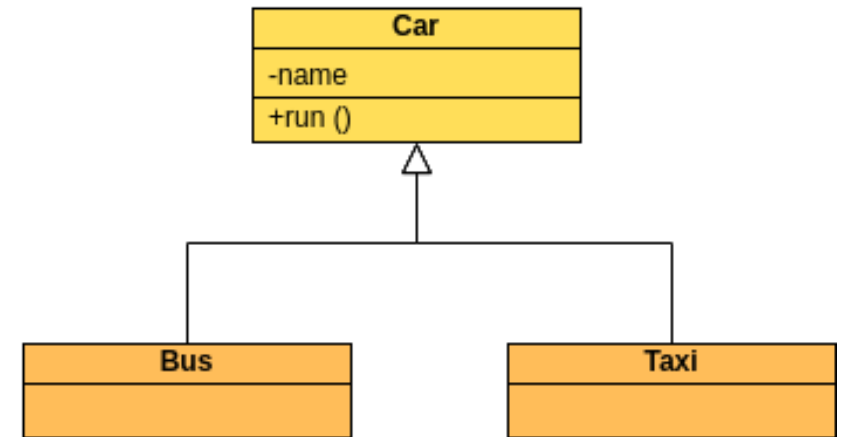
Composición

- La relación entre el todo y la parte, pero el todo y la parte no se pueden separar.
- La existencia de la clase "parte" está estrictamente vinculada a la clase "todo". Si la clase "todo" es destruida, la clase "parte" también lo será, ya que la clase "parte" no puede existir independientemente de la clase "todo"
- Se indica la relación con una línea continua con una punta de rombo negro



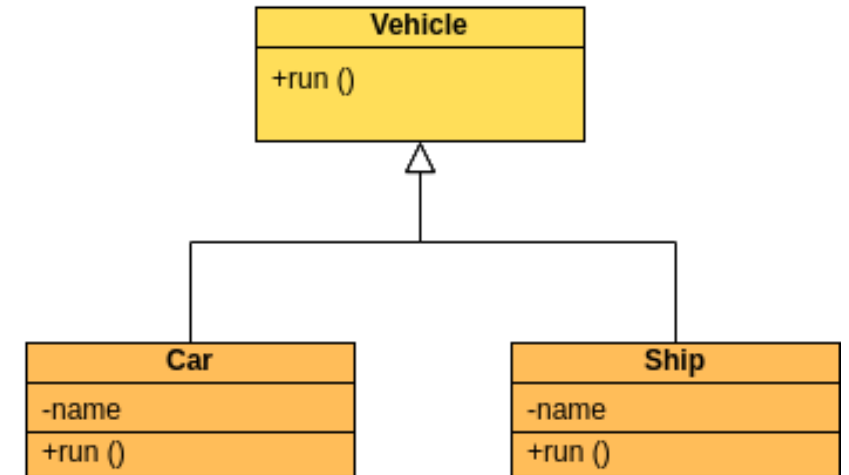
Herencia/Generalización

- Describe la relación entre una clase padre y una clase hija.
- La subclase hereda todas las funciones de la clase principal y la clase principal tiene todos los atributos, métodos y subclases. Las subclases contienen información adicional además de la misma información que la clase principal.
- Se indica la relación con una línea continua con punta de flecha triangular.



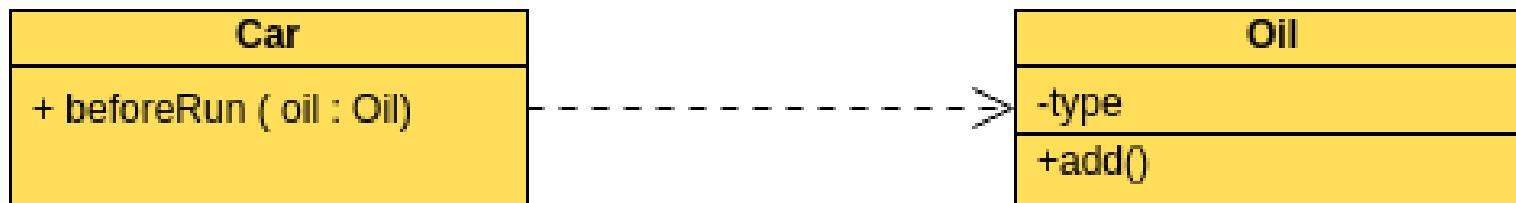
Realización/Implementación

- Describe la relación entre una interface y las clases de implementación.
- **Una interface** (incluida una **clase abstracta**) es una colección de métodos. En una relación de implementación, una clase implementa una interface y los métodos de la clase implementan todos los métodos de la declaración de la interfaz.
- Se indica la relación con una línea continua con punta de flecha triangular (igual que la herencia)



Dependencias

- En la mayoría de los casos, **las dependencias se reflejan en los métodos de una clase que utilizan el objeto de otra clase como parámetro** .
- Una relación de dependencia es una relación de “uso”. Un cambio en una cosa en particular puede afectar a otras cosas que la usan, y usar una dependencia cuando es necesario indicar que una cosa usa otra.
- Se indica la relación con una línea punteada con una punta de flecha.



Pero profe... ¿cómo elijo una relación?



Como elegir relaciones: árbol de decisión

1. ¿La clase hereda atributos o comportamiento de otra?
 - a) Si: usar **Herencia/Generalización**
 - b) No: pasar al (2)
2. ¿La clase implementa métodos de una interface?
 - a) Si: usar **Realización/Implementación**
 - b) No: pasar al (3)
3. ¿La clase necesita de otra para ejecutar un método puntual, de forma momentánea y sin guardarla (no la usa como atributo)?
 - a) Si: usar **Dependencia**
 - b) No: pasar al (4)

Como elegir relaciones: árbol de decisión

4. ¿La clase está relacionada de forma estable con otra (ejemplo, tiene una referencia a ella como atributo)?
 - a) Si: es un tipo de **Asociación**, seguimos con el paso (5)
 - b) No: no hay relación posible.
5. ¿Una clase forma parte de otra (relación todo-parte)?
 - a) Si: Esa parte, ¿puede existir sin el todo?
 - I. Si: usar **Agregación**
 - II. No: usar **Composición**
 - b) No: es una **Asociación** simple

NOTA: se adjunta una versión gráfica en el aula virtual

Ejemplo 1

En un sistema de gestión académica se desean guardar las distintas materias que se dictan y los estudiantes que participan de las materias. Al ser cargado al sistema, los estudiantes aun no participan de ninguna materia. Luego de la matriculación, los estudiantes se inscriben a una o más materias, y cada materia poseerá una lista de estudiantes inscriptos a la misma.

Eligiendo la relación

- 1: ¿Una clase hereda de otra?
 - **No**. Estudiante no es un tipo de Materia, ni viceversa.
- 2: ¿Una clase implementa una interface?
 - **No**. No hay interfaces involucradas.
- 3: ¿Una clase necesita a otra para ejecutar un método puntual?
 - **No**. La relación es más estable: la materia tiene una lista de estudiantes.
- 4: ¿Una clase está relacionada de forma estable con otra?
 - **Sí**, Materia mantiene una referencia a múltiples Estudiantes.
- 5: ¿Es una relación todo-parte?
 - **Sí**. Una Materia está compuesta por varios Estudiantes.
 - ¿La parte puede existir sin el todo?
 - **Sí**. Un Estudiante puede existir incluso si no está inscripto en ninguna Materia.
- Resultado: **Agregación**

Ejemplo 2

En la biblioteca de la institución existen diversos libros que pueden ser gestionados por los bibliotecarios. Un libro puede ser gestionado por uno o más bibliotecarios, dado que ninguno tiene la exclusividad sobre un grupo de libros o libros individuales. A su vez, los bibliotecarios pueden estar encargados de 1, muchos o de ningún libro, según el tipo de tareas al cual esté afectado.

Eligiendo la relación

- 1: ¿Una clase hereda de otra?
 - **No**. Libro no es un tipo de Bibliotecario, ni viceversa.
- 2: ¿Una clase implementa una interface?
 - **No**. No hay interfaces involucradas.
- 3: ¿Una clase necesita a otra para ejecutar un método puntual?
 - **No**. La relación es más estable: el Bibliotecario gestiona información de los Libros.
- 4: ¿Una clase está relacionada de forma estable con otra?
 - **Sí**, un Bibliotecario tiene una relación con varios Libros.
- 5: ¿Es una relación todo-parte?
 - **No**. El Libro no forma parte del Bibliotecario ni dependen del mismo para su existencia.
- Resultado: **Asociación**

Cardinalidades

- La cardinalidad indica **cuántas instancias de una clase pueden estar relacionadas con instancias de otra clase.**
- Permite modelar con mayor precisión la cantidad de objetos involucrados en una relación específica.
- Surge como resultado a las preguntas:
 - ¿Cuántos objetos de la clase A puede tener o usar un objeto de la clase B?
 - ¿Cuántos objetos de la clase B puede tener o usar un objeto de la clase A?

Relaciones con cardinalidad

- Asociación: Si
- Agregación: Si
- Composición: Si
- Dependencias: opcional (no es común)
- Herencia: No *
- Realización: No *

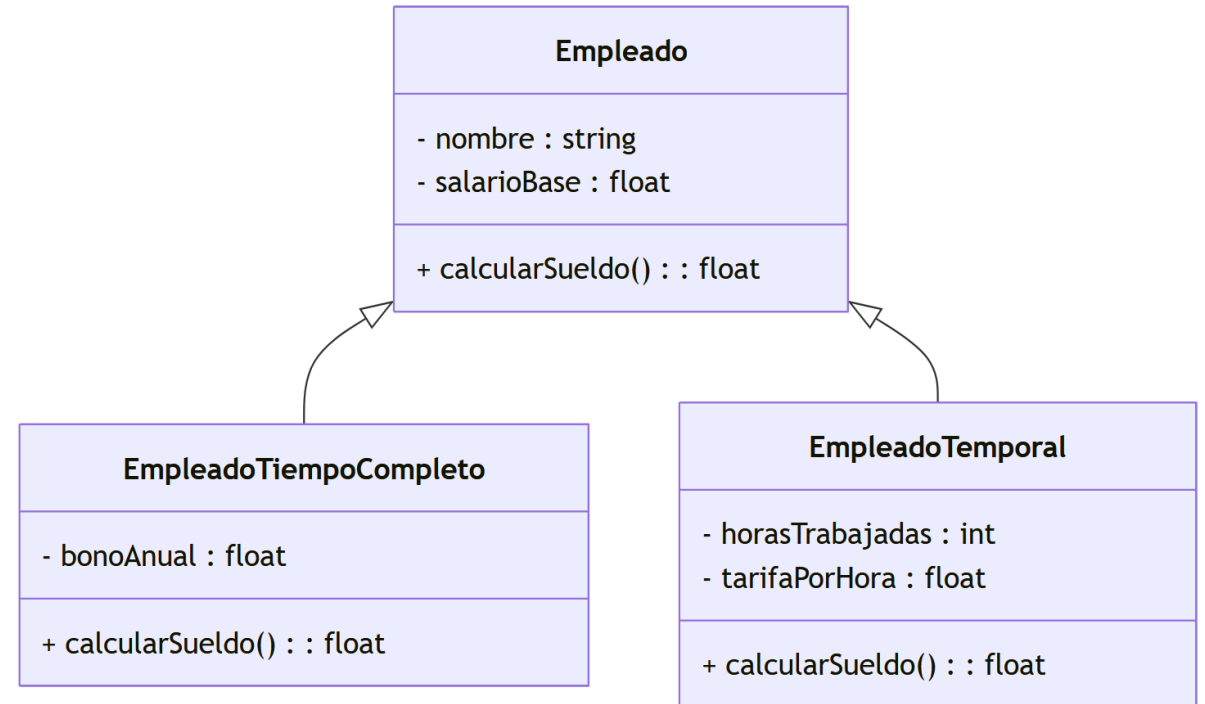
* Herencia y Realización son relaciones de tipo “es un”, no de cantidad.

Tipos de cardinalidades

Notación	Descripción	Ejemplo
1	Exactamente uno	Un <i>Pasaporte</i> pertenece a una <i>Persona</i>
0..1	Cero o uno	Un <i>Empleado</i> puede tener o no un <i>Auto</i>
* (0..*)	Cero o más	Un <i>Curso</i> puede tener muchos <i>Estudiantes</i>
1..*	Uno o más	Una <i>Venta</i> debe tener al menos un <i>Producto</i>
n	Exactamente n (número puntual)	Un <i>Trimestre</i> tiene exactamente 3 <i>Meses</i>
m..n	Desde m hasta n	Un <i>Equipo</i> debe tener entre 5 y 11 <i>Jugadores</i>

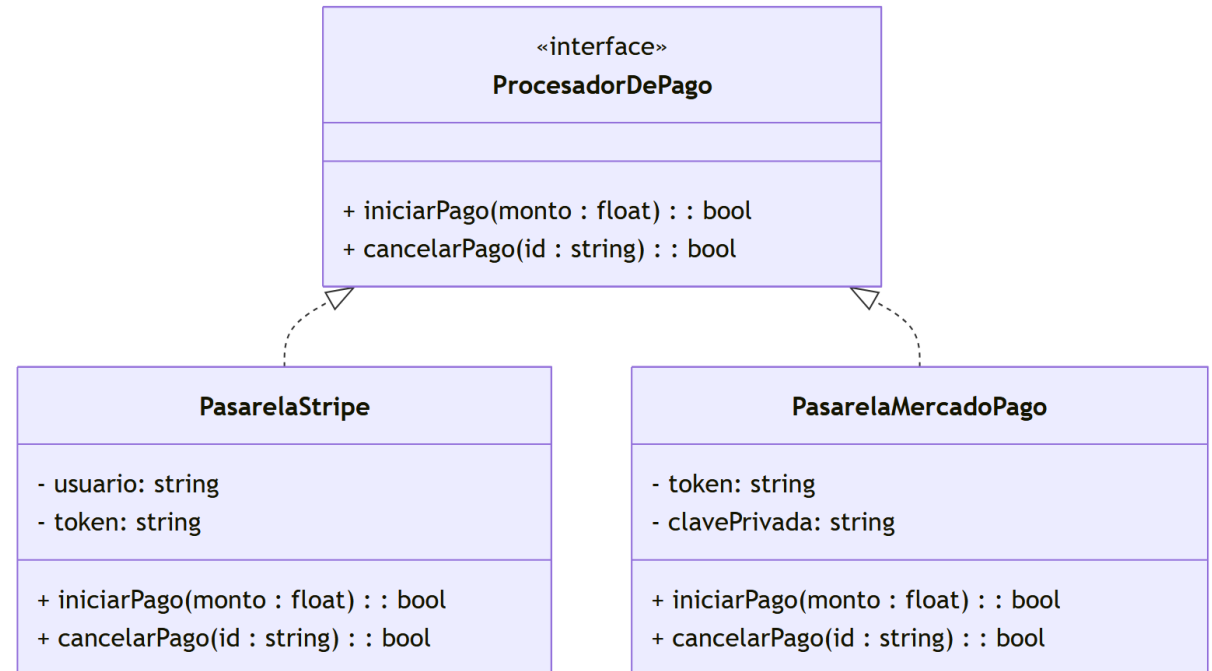
Ejemplo: Herencia

- *EmpleadoTiempoCompleto es un tipo de Empleado* → se representa como herencia directa (sin cardinalidades).
- *EmpleadoTemporal es un tipo de Empleado* → se representa como herencia directa (sin cardinalidades)



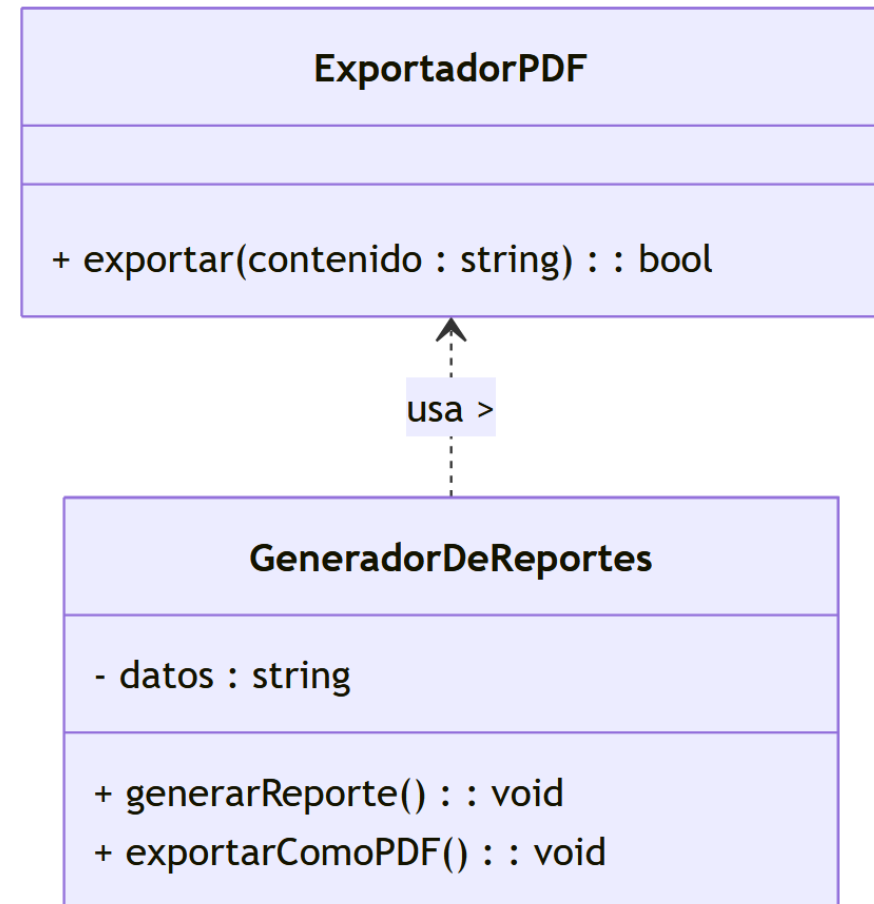
Ejemplo: Realización

- *PasarelaStripe* implementa la interfaz *ProcesadorDePago* → se representa como una realización.
- *PasarelaMercadoPago* implementa la interfaz *ProcesadorDePago* → se representa como una realización.
- No hay cardinalidad, ya que la realización es una relación de comportamiento, no de cantidad.



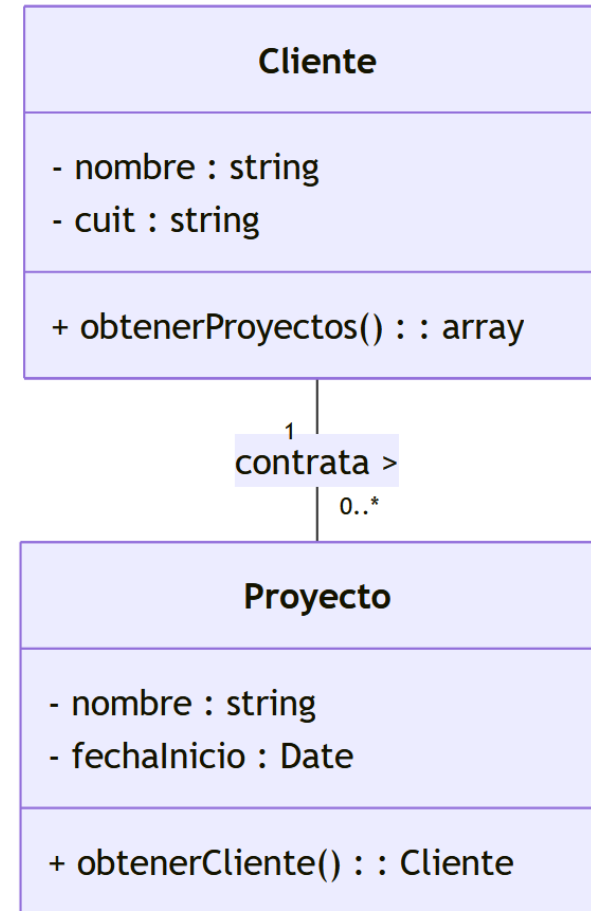
Ejemplo: Dependencia

- En un sistema de generación de reportes, existe una clase *GeneradorDeReportes* que necesita usar temporalmente una instancia de la clase *ExportadorPDF* para exportar los datos del reporte a un archivo PDF.
- *GeneradorDeReportes* depende de *ExportarPDF* → se representa como una dependencia.
- La dependencia no tiene cardinalidad.



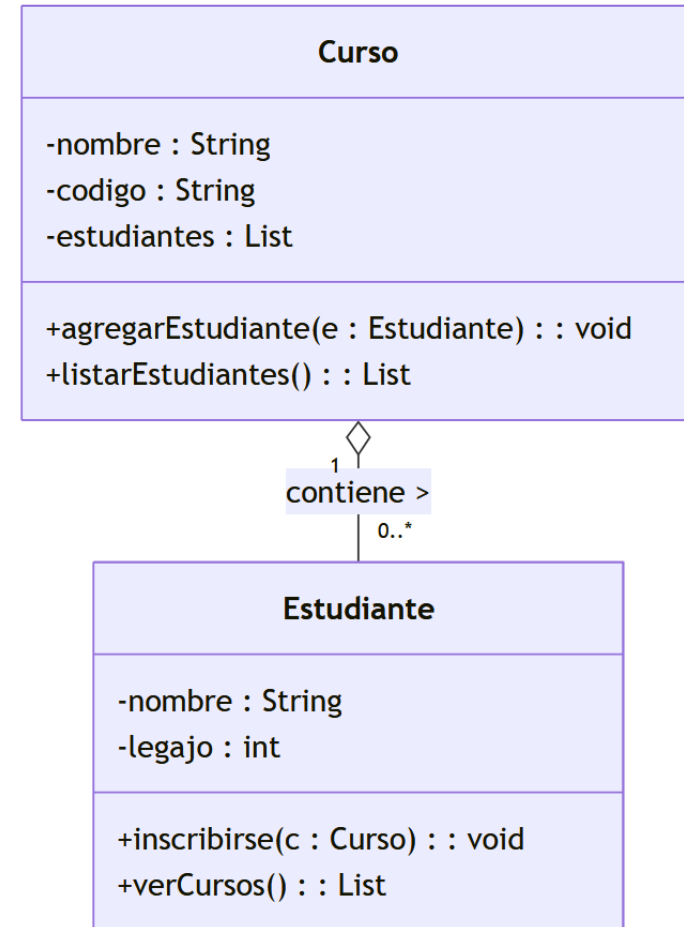
Ejemplo: Asociación

- Un *Cliente* puede tener varios *Proyectos* → se agrega la cardinalidad **0..*** en el extremo de la clase Proyecto.
- Un *Proyecto* tiene un único *Cliente* → se agrega la cardinalidad **1** en el extremo de la clase Cliente.



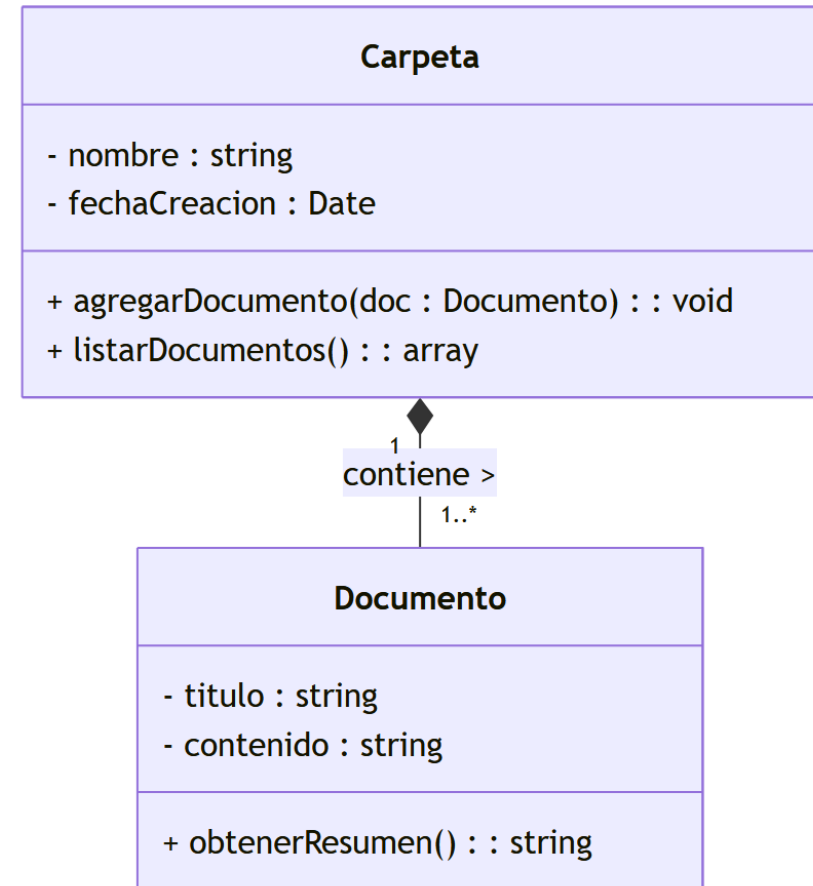
Ejemplo: Agregación

- *Un curso tiene cero o más estudiantes* → se agrega la cardinalidad **0..*** en el extremo de la clase Estudiante.
- *Un estudiante está en un solo curso* → se agrega la cardinalidad **1** en el extremo de la clase Curso.



Ejemplo: Composición

- Una *Carpeta* contiene uno o más *Documentos* → se agrega la cardinalidad **1..*** en el extremo de la clase Documento.
- Un *Documento* pertenece a una sola *Carpeta* → se agrega la cardinalidad **1** en el extremo de la clase Carpeta.



Diagramas de objetos

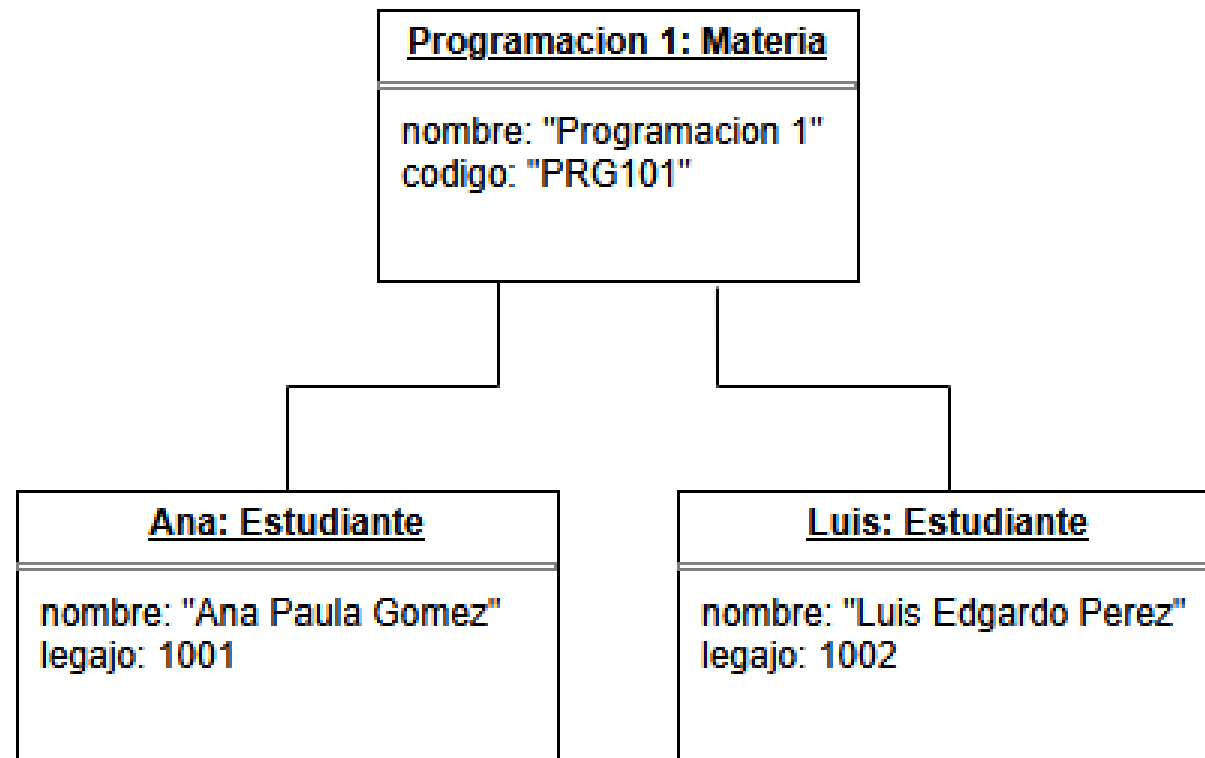
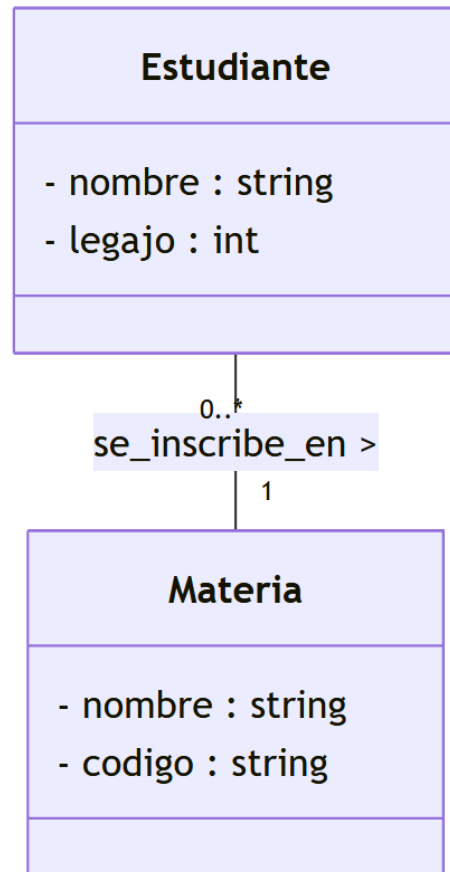
Definición y características

- Un diagrama de objetos en UML es una representación gráfica que muestra un conjunto de **instancias específicas** de clases (objetos) y las **relaciones que existen entre ellos** en un momento determinado del tiempo.
- A diferencia del **diagrama de clases**, que modela la estructura general y abstracta del sistema (clases, atributos, métodos y relaciones), el **diagrama de objetos** se centra en un **estado concreto** del sistema, representando ejemplos reales de objetos con sus valores actuales.
- Para las relaciones usamos solamente **líneas continuas sin puntas** y no se utilizan los indicadores de cardinalidad.
- La línea recta simplemente indica que la relación existe, pero no indica cómo fue definida.

Utilidad

- **Validar el modelo de clases**, verificando si las relaciones y estructuras teóricas son consistentes en situaciones reales.
- **Ilustrar escenarios específicos**, como el estado de un sistema durante una ejecución o caso de uso particular.
- **Facilitar la comprensión del comportamiento del sistema**, especialmente durante el análisis o diseño temprano.
- **Documentar pruebas y condiciones iniciales o finales de procesos** (por ejemplo, mostrar el estado del sistema antes y después de un evento importante).

Ejemplo: clase materia-alumno



Ejercicios

1. Revisión del ejercicio 1 (clase pasada)
2. Revisión del ejercicio 2 (clase pasada)
3. Identificar las relaciones con sus cardinalidades y agregarlas a los diagramas del Ejercicio 1
4. Identificar las relaciones con sus cardinalidades y agregarlas a los diagramas del Ejercicio 2
5. Crear un diagrama de objetos para el Ejercicio 1, incluyendo al menos dos objetos por cada clase.
6. Crear un diagrama de objetos para el Ejercicio 2, incluyendo al menos dos objetos por cada clase.